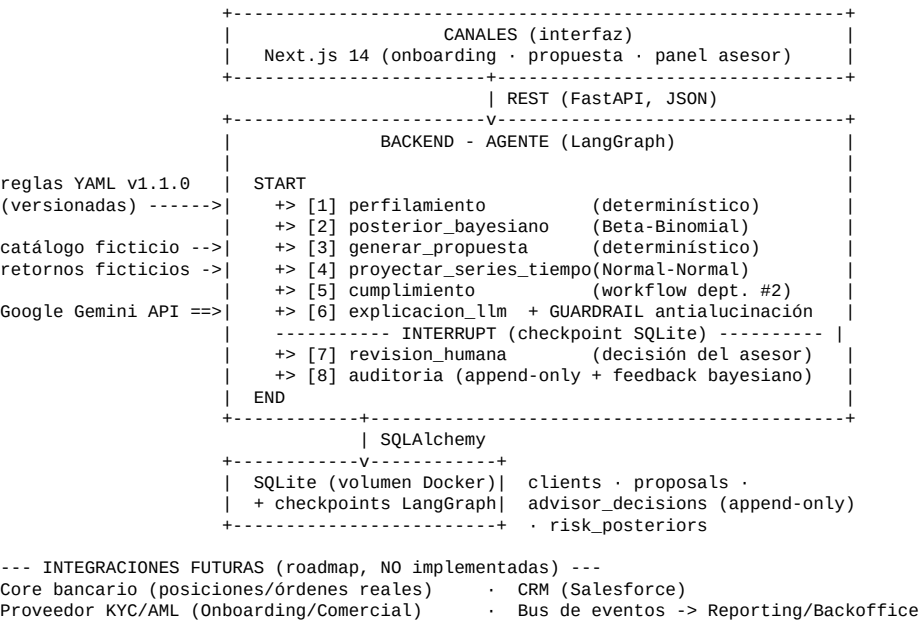


# DOCUMENTO EXPLICATIVO - Odds Ratio

Hackathon Agentic Scale · Ecuador Tech Week 2026 · Club TAWS - ESPOL Equipo: Odds Ratio (Dataclub, ESPOL)  
Track asignado: Track 3 - Robo-Advisory y Automatización de Estrategias de Inversión

Este documento tiene dos objetivos: (1) que el jurado pueda ubicar, línea por línea, dónde vive cada pieza de lógica del sistema, y (2) explicar con rigor estadístico - nuestra carrera de base - cada modelo matemático usado. La IA generativa (Gemini) **solo redacta texto**; todo número que ve el usuario sale de una fórmula estadística versionada y auditable.

## 1. Diagrama de arquitectura



## 2. Track asignado y cumplimiento de reglas obligatorias

Track 3 - Robo-Advisory y Automatización de Estrategias de Inversión.

| Regla obligatoria                                    | Cómo se cumple                                                                    | Dónde en el código                                                            |
|------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Nunca ejecuta órdenes reales ni promete rentabilidad | El grafo se <b>interrumpe</b> antes del nodo final; solo un asesor humano decide. | backend/app/graph/build_graph.py:66-69 (interrupt_before=["revision_humana"]) |
| Datos ficticios / flujo E2E demostrable              | Catálogo e históricos ficticios con semilla fija.                                 | backend/app/data/fictitious_provider.py                                       |
| Disclaimer visible en toda proyección                | Banner en UI + texto del LLM + respuesta JSON.                                    | backend/app/bayes/returns_forecast.py:67-70 (DISCLAIMER)                      |

### 3. Tipo de negocio al que aplica

Instituciones financieras y fintechs con servicio de asesoría de inversión: casas de valores, administradoras de fondos, banca privada / wealth management y fintechs de inversión digital en LATAM. El contexto regulatorio ecuatoriano (Ley de Mercado de Valores) exige responsabilidad humana profesional - Odds Ratio está diseñado alrededor de ese requisito: la **IA prepara y explica**, el **asesor humano autorizado decide**, y cada decisión queda auditada con la versión exacta de reglas y modelos usados.

---

### 4. FUNDAMENTO ESTADÍSTICO - los 4 modelos, en detalle

Esta es la sección central del documento. Cada modelo se explica con sus supuestos, su fórmula, **por qué se eligió sobre alternativas de caja negra** (LSTM/ARIMA/XGBoost/redes neuronales) y **exactamente en qué archivo y función vive el código** que lo calcula.

#### 4.1 Beta-Binomial - confianza en el perfil de riesgo

**Archivo:** backend/app/bayes/risk\_posterior.py (99 líneas) **Versión:** beta-binomial-v1 (constante POSTERIOR\_VERSION, línea 28)

**Problema que resuelve:** el motor de reglas (rules\_engine.py) clasifica al cliente en conservador/moderado/agresivo de forma determinística. Pero, ¿qué tan CONFIABLE es esa clasificación para este cliente en particular? Esa confiabilidad es lo que modelamos como una variable aleatoria bayesiana.

##### Modelo:

Sea  $\theta$  la probabilidad de que la clasificación del motor de reglas sea correcta para un cliente dado, validada por las decisiones del asesor humano.

```
Prior:       $\theta \sim \text{Beta}(\alpha = 2, \beta = 2)$           (líneas 29-30 - débilmente informativo, media 0.5)
Likelihood:  $\text{decisión\_asesor} \sim \text{Bernoulli}(\theta)$ 
Posterior:   $\theta \mid \text{datos} \sim \text{Beta}(\alpha\theta + \text{éxitos}, \beta\theta + \text{fracasos})$ 
```

La familia Beta es el **conjugado natural** de la Binomial/Bernoulli: el posterior tiene forma cerrada, no requiere MCMC ni aproximaciones. Esto es clave para un sistema regulado - cada número se recalcula a mano.

**Retroalimentación (Historia 3)** - función `update_posterior()`, líneas 59-68, con la tabla de actualización DECISION\_UPDATES (líneas 33-37):

```
aprobar ->  $\alpha += 1.0$       (el asesor confirma: la clasificación fue correcta)
rechazar ->  $\beta += 1.0$       (el asesor corrige: la clasificación fue incorrecta)
editar ->  $\alpha += 0.5, \beta += 0.5$  (parcialmente correcta - matiz)
```

**Estadísticos reportados** (función `posterior_summary()`, líneas 89-98):

- **Confianza** =  $E[\theta] = \alpha/(\alpha+\beta)$  - media del posterior (función `confidence()`, línea 71-73).
- **Intervalo de credibilidad al 90 %** - por Monte Carlo (`credible_interval()`, líneas 76-86): se muestrean 20 000 valores de  $\text{Beta}(\alpha, \beta)$  con semilla fija (`rng = np.random.default_rng(7)`) y se toman los percentiles 5 % y 95 %. Se usa Monte Carlo en vez de la inversa analítica de la Beta para no depender de scipy (dependencia mínima,

resultado idéntico hasta el 4º decimal).

**Por qué Beta-Binomial y no una red neuronal de clasificación:** una red aprendería a "corregir" el perfil directamente, pero sería una caja negra que un regulador no puede auditar caso por caso. El posterior Beta es una sola fórmula algebraica - cualquier auditor puede reconstruir "por qué la confianza de este cliente es 63 %" con lápiz y papel:  $\alpha / (\alpha + \beta)$ .

#### Dónde se usa:

- Se crea/lee en el nodo 2 del grafo - backend/app/graph/nodes.py:41-54 (nodo\_posterior\_bayesiano).
- Se actualiza en el nodo 8 - backend/app/graph/nodes.py:258-316 (nodo\_auditoria, líneas 306-309).
- Se expone vía API en GET /review/history y en el ranking bayesiano de clientes (backend/app/bayes/client\_ranking.py, Thompson sampling sobre este mismo posterior).
- Se muestra al asesor en frontend/app/asesor/page.tsx ( $\alpha$ ,  $\beta$  y confianza en vivo tras cada decisión).

---

## 4.2 Normal-Normal conjugado - proyección del portafolio (fan chart)

**Archivo:** backend/app/bayes/returns\_forecast.py (165 líneas) **Versión:** normal-normal-v2-bankpriors (línea 43)

**Problema que resuelve:** proyectar el valor futuro de un portafolio sin prometer una cifra puntual (prohibido por el track) - se necesita una **distribución** de escenarios posibles.

**Modelo, por clase de activo:** el retorno mensual  $r$  se asume  $\text{Normal}(\mu, \sigma^2)$  con  $\sigma^2$  conocida (varianza muestral del histórico ficticio) y un prior conjugado sobre la media  $\mu$ :

```
Prior:       $\mu \sim \text{Normal}(\mu_0, \tau_0^2)$ 
Posterior:   $\mu \mid \text{datos} \sim \text{Normal}(\mu_n, \tau_n^2)$ 

 $\mu_n = (\mu_0/\tau_0^2 + n \cdot \bar{x} / \sigma^2) / (1/\tau_0^2 + n/\sigma^2)$     [precisión ponderada prior vs. datos,  $\bar{x}$  = media muestral]
 $\tau_n^2 = 1 / (1/\tau_0^2 + n/\sigma^2)$ 

Predictiva:  $r_{\text{futuro}} \sim \text{Normal}(\mu_n, \sigma^2 + \tau_n^2)$ 
```

Implementado literalmente así en posterior\_predictive(), líneas 73-101 -  $\text{prec\_prior} = 1/\tau_0^2$ ,  $\text{prec\_data} = n/\sigma^2$ ,  $\mu_n = (\mu_0 \cdot \text{prec\_prior} + n \cdot \bar{x} / \sigma^2) / (\text{prec\_prior} + \text{prec\_data})$  (línea 89). Es la fórmula de **actualización bayesiana con precisión aditiva**: cuantos más datos históricos ( $n$  grande), más pesa  $\bar{x}$  (la media muestral) sobre el prior  $\mu_0$ .

**Priors informados por la práctica bancaria real** (no arbitrarios): un banco no deja el dinero de sus clientes estancado - mantiene un encaje (~5 %) y reinvierte el resto (~75 %) en cartera de crédito, ganando el spread entre la tasa activa (~9.5 % anual, lo que cobra al prestar) y la tasa pasiva (~5.5 % anual, lo que paga al captar). Cada clase de activo ancla su  $\mu_0/\tau_0$  a esas tasas reales - ver backend/app/data/bank\_priors\_v1.json (versionado, justificado clase por clase) y función prior\_for() (líneas 59-65).

**Simulación Monte Carlo del portafolio** - función forecast\_portfolio(), líneas 104-164:

1. Se combina  $\mu$  y  $\sigma^2$  de cada clase según los pesos de la propuesta (líneas 125-135, con la simplificación documentada de independencia entre clases - MVP).
2. Se generan 4 000 trayectorias mensuales con semilla fija (seed=123, línea 137) vía rng.normal(...) y se acumulan con np.cumprod(1 + retornos) (línea 139) - así el capital compone mes a mes, como una inversión real.

3. Para cada mes se calculan los **percentiles 5/25/50/75/95** (líneas 141-153) -> esto es el **fan chart**: nunca una línea, siempre un abanico de escenarios (p05 = pesimista, p50 = mediana, p95 = optimista).

**Por qué Normal-Normal y no ARIMA/LSTM para pronosticar series de tiempo:** ARIMA y LSTM buscan CAPTURAR AUTOCORRELACIÓN y patrones no lineales del precio - son mejores prediciendo la serie en sí. Pero acá el objetivo no es "adivinar el precio de mañana": es comunicar honestamente la incertidumbre de un portafolio diversificado a varios años. El modelo conjugado da una distribución cerrada, auditable, con un prior justificable por la economía real del negocio bancario - no un ajuste de hiperparámetros de una red.

**Dónde se usa:**

- Nodo 4 del grafo - backend/app/graph/nodes.py:89-99 (nodo\_proyectar\_series\_tiempo).
- Endpoint POST /forecast (backend/app/routers/forecast.py:13).
- Frontend: fan chart en frontend/app/propuesta/page.tsx (Recharts, área apilada p05-p95 + línea p50).

**4.3 Markowitz (Media-Varianza) - Teorema de Separación de Dos Fondos**

**Archivo:** backend/app/bayes/portfolio\_optimizer.py (161 líneas) **Versión:** markowitz-mv-v2-twofund (línea 53)

**Problema que resuelve:** validar la asignación oficial (por reglas transparentes) contra lo que diría la **Teoría Moderna de Carteras** (Markowitz, 1952) - maximizar retorno esperado para un nivel de riesgo dado, usando la covarianza real entre clases de activo. Este modelo **no reemplaza** la asignación oficial; la complementa como segunda opinión cuantitativa.

**Insumos - ningún número nuevo se inventa:**

- $\mu$  (**retornos esperados**): el MISMO posterior Normal-Normal de la sección 4.2 (expected\_returns( ), líneas 76-78, llama a returns\_forecast.posterior\_predictive()).
- $\Sigma$  (**matriz de covarianza**): calculada empíricamente con np.cov( ) (línea 90) sobre los mismos retornos históricos ficticios que usa todo el sistema.

**Por qué Separación de Dos Fondos y no un  $\Sigma^{-1}\mu$  ingenuo sobre las 5 clases:** "efectivo" actúa como proxy del activo libre de riesgo - su varianza histórica es casi cero. Invertir una matriz de covarianza completa que incluya un activo de varianza ~0 la vuelve numéricamente casi singular (mal condicionada) y el resultado deja de tener sentido económico (lo detectamos en pruebas internas: la optimización ingenua asignaba ~95 % a efectivo sin importar el perfil). Por eso aplicamos el teorema clásico de Tobin (1958) / Sharpe (1964):

```
Paso 1 - Cartera tangente (solo activos riesgosos, SIN efectivo):
w_tan ~ Σ_riesgo^-1 (μ_riesgo - r_f)          long-only, recortada a >= 0
-> función tangency_portfolio(), líneas 94-120
-> resuelto con np.linalg.solve(sigma, exceso), línea 108 (evita invertir Σ explícitamente)

Paso 2 - Mezcla óptima riesgo/efectivo según aversión al riesgo λ:
y = (μ_tan - r_f) / (λ · σ²_tan),   recortado a [0, 1] (sin apalancar, sin cortos)
cartera final = y · w_tan + (1 - y) · efectivo
-> función optimize_for_profile(), líneas 123-160
```

$\lambda$  (aversión al riesgo) se ANCLA al perfil ya calculado por las reglas transparentes - no es una preferencia libre del modelo:

| Perfil      | $\lambda$ | Efecto                             |
|-------------|-----------|------------------------------------|
| conservador | 200       | y pequeño -> casi todo en efectivo |
| moderado    | 80        | mezcla intermedia                  |

| Perfil   | $\lambda$ | Efecto                                                         |
|----------|-----------|----------------------------------------------------------------|
| agresivo | 40        | y -> 1 -> satura la cartera tangente (100 % activos riesgosos) |

(tabla RISK\_AVERSION, líneas 61-65 - calibrada a la escala real de la varianza mensual de los retornos ficticios,  $\sim 1e-5$ , para que y no sature siempre en 0 o 1).

**Por qué Markowitz y no XGBoost/deep learning para asignar la cartera:** en asesoría regulada, un regulador puede exigir "explique por qué este cliente recibió esta cartera". Markowitz es una optimización cuadrática clásica con solución cerrada - se reconstruye con álgebra lineal de pregrado. Un modelo de ensamble o una red neuronal sería una caja negra que no se puede justificar número por número ante un cliente o un regulador.

**Dónde se usa:**

- Endpoint GET /forecast/markowitz/{perfil} (backend/app/routers/forecast.py:38).
- Frontend: tarjeta comparativa "Markowitz vs. Reglas" en frontend/app/propuesta/page.tsx.

#### 4.4 Alfa de Cronbach - consistencia interna del cuestionario

**Archivo:** backend/app/analytics/quality\_report.py (185 líneas) **Versión:** quality-report-v1 (línea 31)

**Problema que resuelve:** con 10 preguntas obligatorias que puntúan de forma aditiva hacia un mismo score de riesgo, necesitamos verificar que realmente miden **un mismo constructo latente** (la tolerancia/capacidad de riesgo) y no ruido. Ese es el propósito clásico del Alfa de Cronbach.

**Fórmula** (implementada literalmente en cronbach\_alpha(), líneas 35-75):

$$\alpha = (k / (k - 1)) \cdot (1 - \sum \text{Var}(\text{item}_i) / \text{Var}(\text{total}))$$

donde  $k$  = número de ítems (10),  $\text{Var}(\text{item}_i)$  = varianza de los puntos de cada pregunta entre todos los clientes,  $\text{Var}(\text{total})$  = varianza de la suma de puntos por cliente. Código: línea 63 (`var_items = matriz.var(axis=0, ddof=1)`), línea 64 (`var_total = matriz.sum(axis=1).var(ddof=1)`), línea 68 (fórmula final). Se usa `ddof=1` (varianza muestral, no poblacional) porque cada cuestionario es una muestra de la población de inversionistas.

**Salvaguardas metodológicas explícitas en el código:**

- Se filtra por `rules_version` vigente (línea 44) para no mezclar cuestionarios de distintas versiones (escalas de puntos distintas invalidan la comparación de varianzas).
- Requiere mínimo 2 registros completos y varianza total no nula, si no, devuelve `alfa: None` con una nota explicando por qué (líneas 48-53, 65-66) - nunca se reporta un número no confiable.
- Interpretación estándar de la literatura psicométrica (`_interpretar_alfa()`, líneas 78-87):  $\geq 0.9$  excelente,  $\geq 0.8$  buena,  $\geq 0.7$  aceptable (mínimo para un instrumento de perfilamiento),  $\geq 0.6$  cuestionable,  $< 0.6$  pobre.

**Detección de casos atípicos por z-score** - función `question_timing_stats()`, líneas 90-140: para cada pregunta, se calcula  $z = (\text{tiempo\_respuesta} - \text{media}) / \text{desviación}$  (línea 124) y se marca como "caso crítico" cualquier respuesta con  $|z| > 2.0$  (constante `Z_SCORE_ATIPICO`, línea 32) - señal de que la pregunta puede estar confusa (tiempo anormalmente alto) o de respuesta apresurada sin lectura (tiempo anormalmente bajo). Estos datos solo se capturan en

sesiones de **Modo Piloto** (muestreo intencional / casos críticos), nunca de forma forzada a todos los usuarios.

**Dónde se usa:**

- Endpoint GET /review/quality-report (backend/app/routers/review.py:132, protegido - solo operadores).
- Frontend: pestaña "Calidad" en frontend/app/asesor/page.tsx.

## 5. Conexiones API - cómo se comunican las piezas

### 5.1 Frontend -> Backend (REST/JSON):

Toda la comunicación pasa por un único cliente HTTP centralizado: frontend/lib/api.ts. La URL base se resuelve en build/runtime (línea 10-11):

```
export const API_URL = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";
```

En local apunta a localhost:8000; en Docker/nube se inyecta NEXT\_PUBLIC\_API\_URL como variable de entorno al construir la imagen - sin tocar el código. Cada función exportada (createProposal, getHistory, sendDecision, getQualityReport, etc.) hace un fetch() tipado hacia un endpoint específico del backend; los tipos TypeScript (interfaces Proposal, FanChartPoint, AllocationItem...) son un espejo manual de los schemas.py (Pydantic) del backend.

### 5.2 Endpoints REST expuestos (montados en backend/app/main.py):

| Router    | Endpoint                     | Método     | Protegido     |
|-----------|------------------------------|------------|---------------|
| profiling | /profiling/questionnaire     | GET        | No            |
| profiling | /profiling/pilot-status      | GET        | No            |
| profiling | /profiling/evaluate          | POST       | No            |
| proposals | /proposals                   | POST / GET | No            |
| proposals | /proposals/{id}              | GET        | No            |
| forecast  | /forecast                    | POST       | No            |
| forecast  | /forecast/asset-classes      | GET        | No            |
| forecast  | /forecast/markowitz/{perfil} | GET        | No            |
| review    | /review/pending              | GET        | Sí (operador) |
| review    | /review/history              | GET        | Sí (operador) |
| review    | /review/quality-report       | GET        | Sí (operador) |
| review    | /review/{id}/decision        | POST       | Sí (operador) |
| review    | /review/audit                | GET        | Sí (operador) |
| chat      | /chat                        | POST       | No            |

| Router  | Endpoint                | Método  | Protegido     |
|---------|-------------------------|---------|---------------|
| chat    | /chat/logs              | GET     | Sí (operador) |
| clients | /auth/login             | POST    | No            |
| clients | /autopilot, /pilot-mode | GET/PUT | Sí (operador) |
| clients | /clients/ranking        | GET     | Sí (operador) |

Protección: header X-Operator-Token validado por `require_operator` (backend/app/auth.py) contra un hash SHA-256 de las credenciales demo (operador / oddsratio2026, configurables por variables de entorno).

### 5.3 Backend -> LangGraph -> Gemini:

1. POST /proposals llama a `start_advisory()` en backend/app/workflows/asesoria\_workflow.py -> compila/invoca el grafo singleton (`get_graph()`, `build_graph.py:37`) con un `thread_id` nuevo.
2. El grafo ejecuta los nodos 1-6 en secuencia (ver sección 1) hasta el nodo `explicacion_llm` (nodes.py:212-236), que arma un **JSON de solo lectura** (`_resumen_para_llm()`, líneas 186-209) y llama a `provider.generate(prompt)`.
3. `GeminiProvider.generate()` (backend/app/llm/gemini\_provider.py:26-38) invoca el SDK oficial google-genai: `self._client.models.generate_content(model="gemini-2.5-flash", contents=prompt)`. Si Gemini falla (cuota, red) o devuelve vacío, cae automáticamente a `MockLLMProvider` (misma interfaz `LLMProvider`) para que la demo nunca se rompa.
4. La respuesta de Gemini pasa por `validar_consistencia_numerica()` (nodes.py:171-183): extrae todos los números del texto con regex y los compara contra los números presentes en el JSON de entrada (con tolerancia de redondeo). Si hay una sola cifra "extraña", se descarta la respuesta y se usa la plantilla determinística del mock - el evento queda registrado (`guardrail_activated: true` en la auditoría).
5. El grafo se **interrumpe** (`interrupt_before=["revision_humana"]`, `build_graph.py:68`) y el checkpoint queda en SQLite (`langgraph_checkpoints.db`) hasta que el asesor decide.
6. POST /review/{id}/decision llama a `resume_advisory()` (asesoria\_workflow.py) -> reanuda el MISMO `thread_id`, ejecuta `revision_humana` y `auditoria` (nodes.py:244-316), actualiza el posterior Beta-Binomial (sección 4.1) y escribe el registro `append-only`.

**5.4 Persistencia:** SQLAlchemy + SQLite (backend/app/database.py). Tablas clave (backend/app/models.py): `Client`, `RiskProfileRecord`, `RiskPosterior`, `Proposal`, `SystemSetting`, `ChatMessage`, `AdvisorDecision` (`append-only` - nunca se actualiza ni borra un registro, solo se agregan nuevos).

## 6. Cómo se integraría a un sistema empresarial existente

1. **API REST como contrato:** un core bancario (Temenos, Cobis, etc.) consumiría los mismos endpoints que hoy usa el frontend.
2. **CRM (Salesforce):** propuestas y decisiones se sincronizan como actividades del cliente vía webhooks/REST.
3. **KYC/AML:** el nodo de perfilamiento consumiría el resultado de un proveedor de verificación de identidad sin cambiar el grafo.

4. **Bus de eventos (Kafka/RabbitMQ):** cada transición del grafo emitiría eventos (`propuesta.creada`, `decision.registrada`) para Cumplimiento y Reporting/Backoffice.
5. **Datos de mercado reales:** la interfaz `DataProvider` (`backend/app/data/provider.py`) está diseñada para sustituir el proveedor ficticio por Bloomberg/Refinitiv sin tocar la lógica de negocio.
6. **Identidad:** el rol "asesor autorizado" se conectaría a OAuth2/SAML corporativo en vez del token demo actual.

---

## 7. Evidencia de pruebas automatizadas

`backend/tests/` - **65 tests, todos pasando** (`python -m pytest tests/ -v`):

| Archivo                                  | Qué prueba                                                                                                          |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>test_rules_engine.py</code>        | Perfilamiento: umbrales, influencias, versiones, 10 preguntas obligatorias, validación de entradas.                 |
| <code>test_bayes_posterior.py</code>     | Beta-Binomial: prior, actualización por decisión, intervalo de credibilidad, reproducibilidad.                      |
| <code>test_portfolio_optimizer.py</code> | Markowitz: cartera tangente, pesos no negativos, aversión al riesgo por perfil.                                     |
| <code>test_graph_nodes.py</code>         | Nodos LangGraph con LLM mockeado: guardrail antialucinación, flujo E2E interrupt->resume, doble decisión bloqueada. |
| <code>test_chat_ranking_auth.py</code>   | Chat con guardrail de cifras, login/token de operadores, ranking bayesiano, priors bancarios.                       |
| <code>test_autopilot.py</code>           | Modo Autopiloto: umbral, auto-aprobación firmada, alertas de Cumplimiento revocan autonomía.                        |
| <code>test_muestreo.py</code>            | Estratos A/B/C, Modo Piloto, Alfa de Cronbach real, casos críticos por z-score.                                     |
| <code>test_review_history.py</code>      | Las 3 Historias consolidadas para el asesor.                                                                        |

---

## 8. Mitigación de riesgos / antialucinación (resumen)

- Todo número sale de **reglas determinísticas o modelos estadísticos versionados** (secciones 4.1-4.4) - el LLM jamás calcula, solo redacta.
- Guardrail activo y verificado por test: `nodes.py:171-183`.
- Auditoría append-only con snapshot completo y versión exacta de reglas y modelo bayesiano usados en cada decisión.
- Disclaimers visibles en UI, API y texto generado.